Machine Learning Final Project Code

Shuman Qi sq2087 & Boyuan Zhang bz2118

```
pkgs <- c("ggplot2", "caret", "randomForest",
          "corrplot", "nnet", "dplyr")
for (p in pkgs) {
  if (!requireNamespace(p, quietly = TRUE)) install.packages(p)
}
library(ggplot2)
library(caret)
```

```
## Loading required package: lattice
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(nnet)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following object is masked from 'package:randomForest':
##
##     combine
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(xgboost)
```

```
##
## Attaching package: 'xgboost'
```

```
## The following object is masked from 'package:dplyr':
##
##     slice
```

```
set.seed(42)
df <- read.csv("/Users/shermanqi/Desktop/2338/final project/Social_media_impact_on_life.csv", stringsAs
print(names(df))
```

```
##  [1] "Student_ID"                  "Age"
##  [3] "Gender"                      "Academic_Level"
##  [5] "Country"                     "Avg_Daily_Usage_Hours"
##  [7] "Most_Used_Platform"          "Affects_Academic_Performance"
##  [9] "Sleep_Hours_Per_Night"       "Mental_Health_Score"
## [11] "Overall_Impact"
```

```
print(head(df, 5))
```

```
##   Student_ID Age Gender Academic_Level Country Avg_Daily_Usage_Hours
## 1        232  21   Male  Undergraduate   Other                   4.0
## 2        564  23 Female  Undergraduate   Other                   1.6
## 3        788  22   Male       Graduate  Canada                   4.6
## 4        686  18   Male  Undergraduate   Other                   7.0
## 5        608  24 Female    High School   Other                   7.5
##   Most_Used_Platform Affects_Academic_Performance Sleep_Hours_Per_Night
## 1           Facebook                          No                   6.7
## 2           LinkedIn                          No                   8.6
## 3          Instagram                          No                   6.7
## 4           Snapchat                         Yes                   5.4
## 5           Facebook                         Yes                   5.0
##   Mental_Health_Score Overall_Impact
## 1                 6.8        Neutral
## 2                 7.6       Positive
## 3                 7.0        Neutral
## 4                 5.3       Negative
## 5                 4.4       Negative
```

```
print(colSums(is.na(df)))
```

```
##                  Student_ID                         Age
##                           0                           0
##                      Gender              Academic_Level
##                           0                           0
```

```
##                       Country        Avg_Daily_Usage_Hours
##                             0                            0
##           Most_Used_Platform Affects_Academic_Performance
##                             0                            0
##        Sleep_Hours_Per_Night          Mental_Health_Score
##                             0                            0
##               Overall_Impact
##                             0
```

```
print(table(df$Overall_Impact))
```

```
##
## Negative  Neutral Positive
##      939      267      499
```

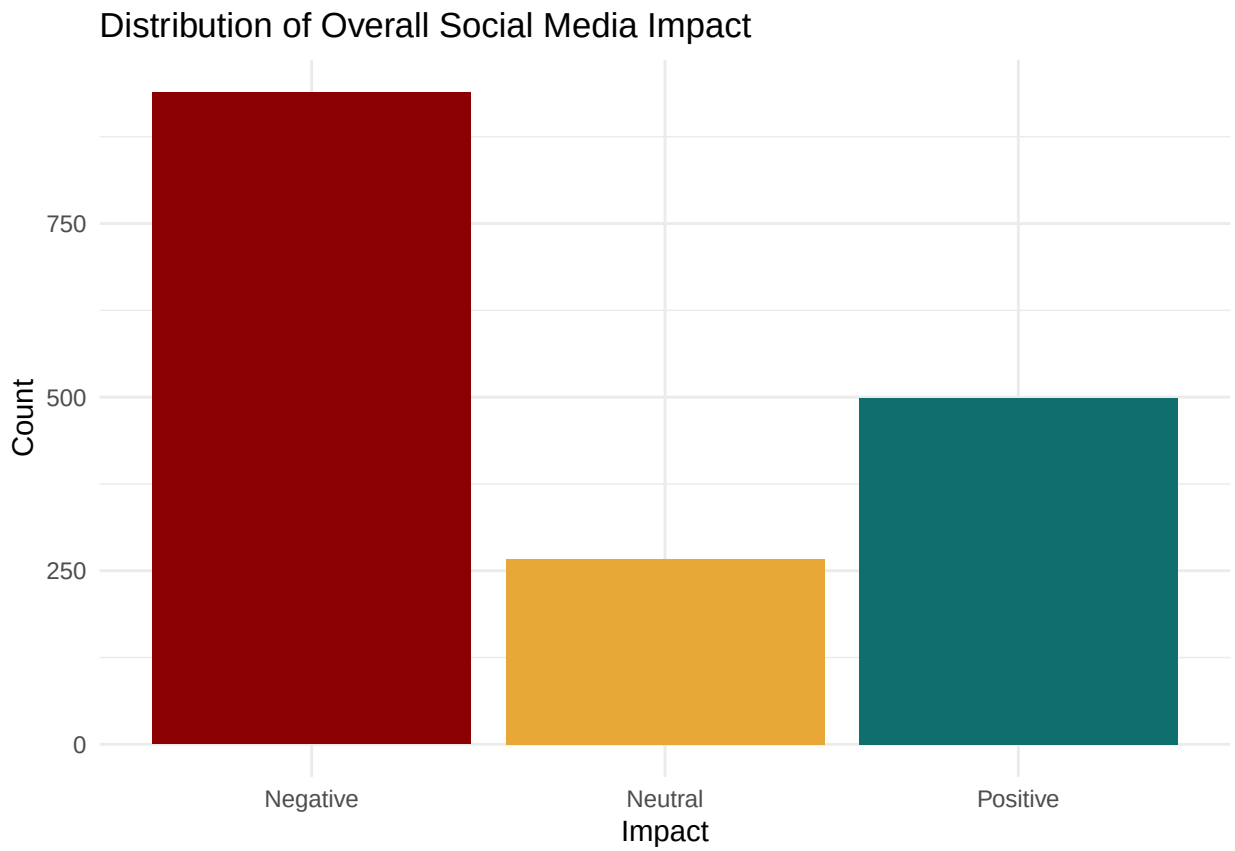
```
print(round(prop.table(table(df$Overall_Impact)) * 100, 1))
```

```
##
## Negative  Neutral Positive
##     55.1     15.7     29.3
```

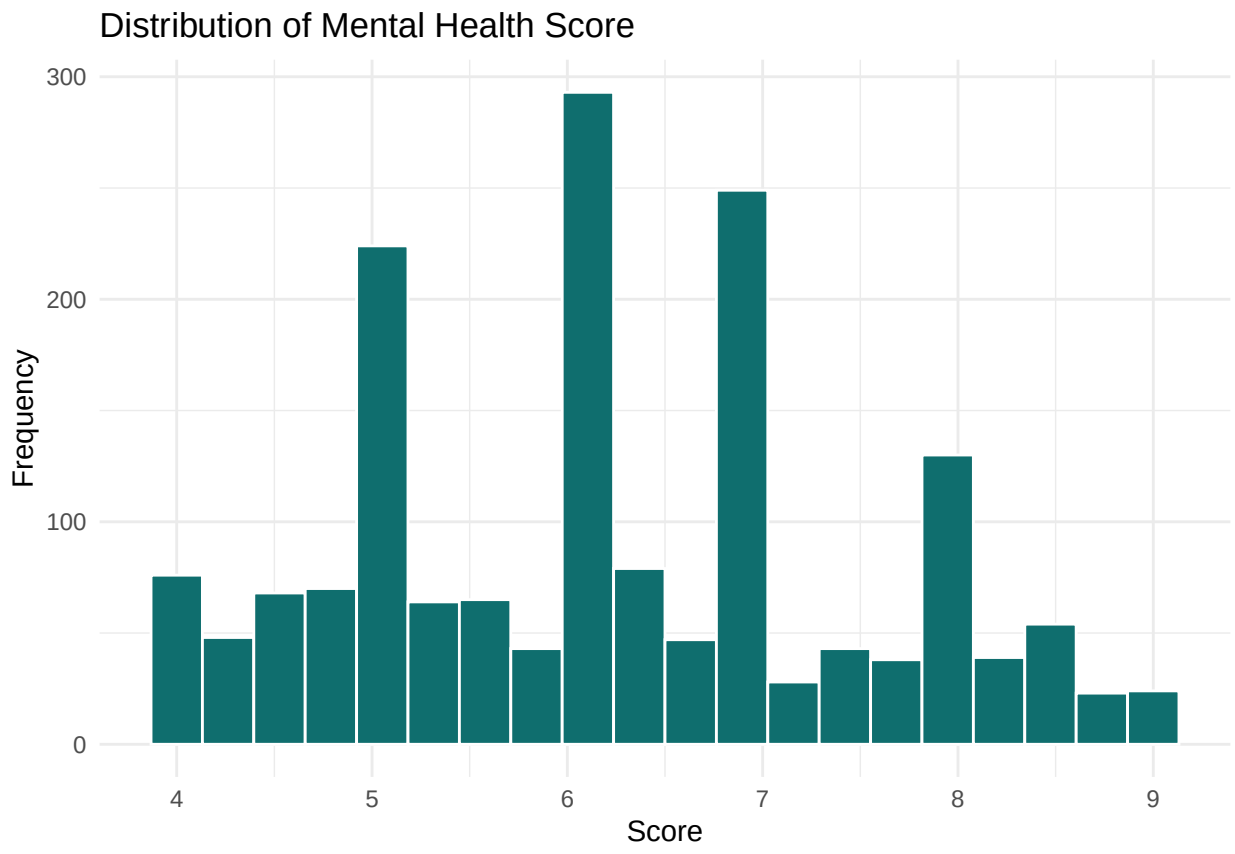
```
print(summary(df$Mental_Health_Score))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   4.000   5.000   6.000   6.215   7.000   9.000
```

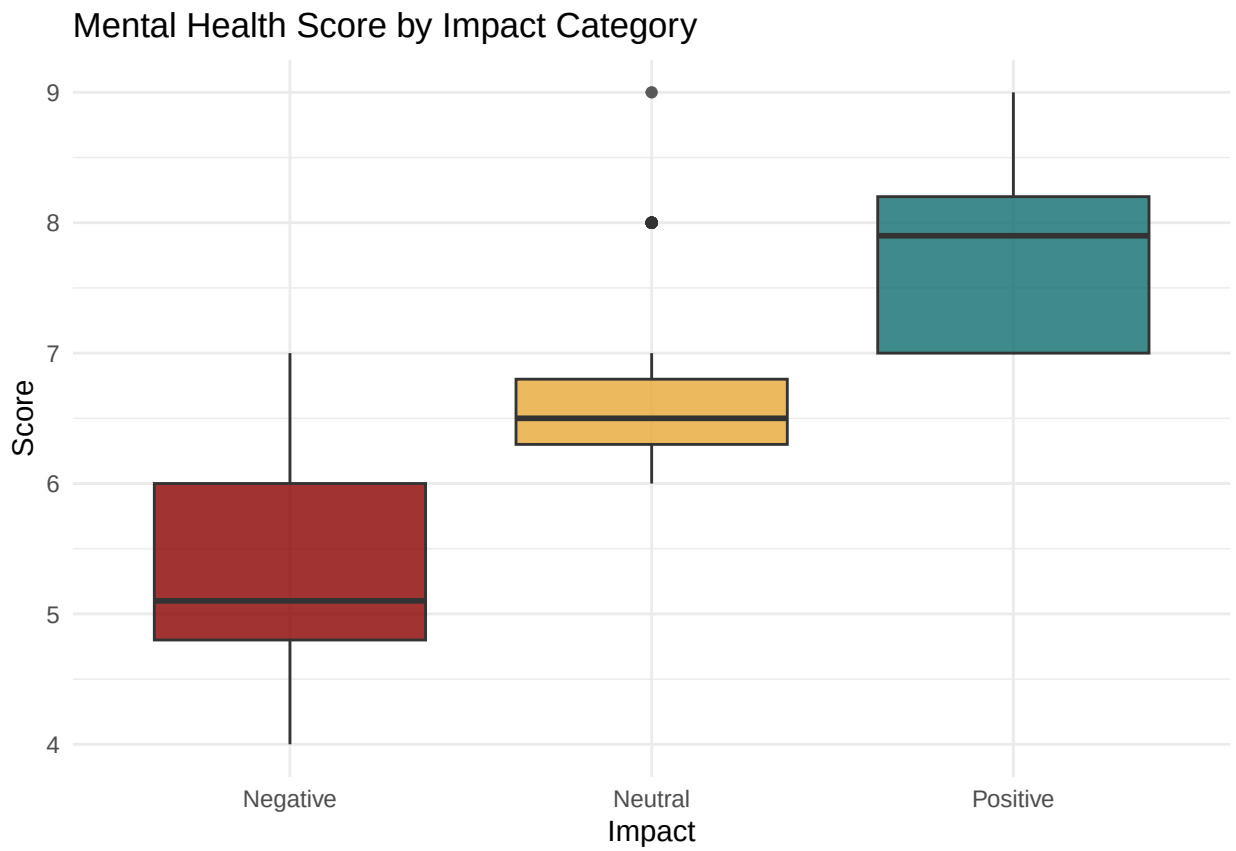
```
p1 <- ggplot(df, aes(x = Overall_Impact, fill = Overall_Impact)) +
  geom_bar() +
  scale_fill_manual(values = c("Positive" = "#0F6E6E",
                               "Neutral"  = "#E8A838",
                               "Negative" = "#8B0000")) +
  labs(title = "Distribution of Overall Social Media Impact",
       x = "Impact", y = "Count") +
  theme_minimal() + theme(legend.position = "none")
print(p1)
```



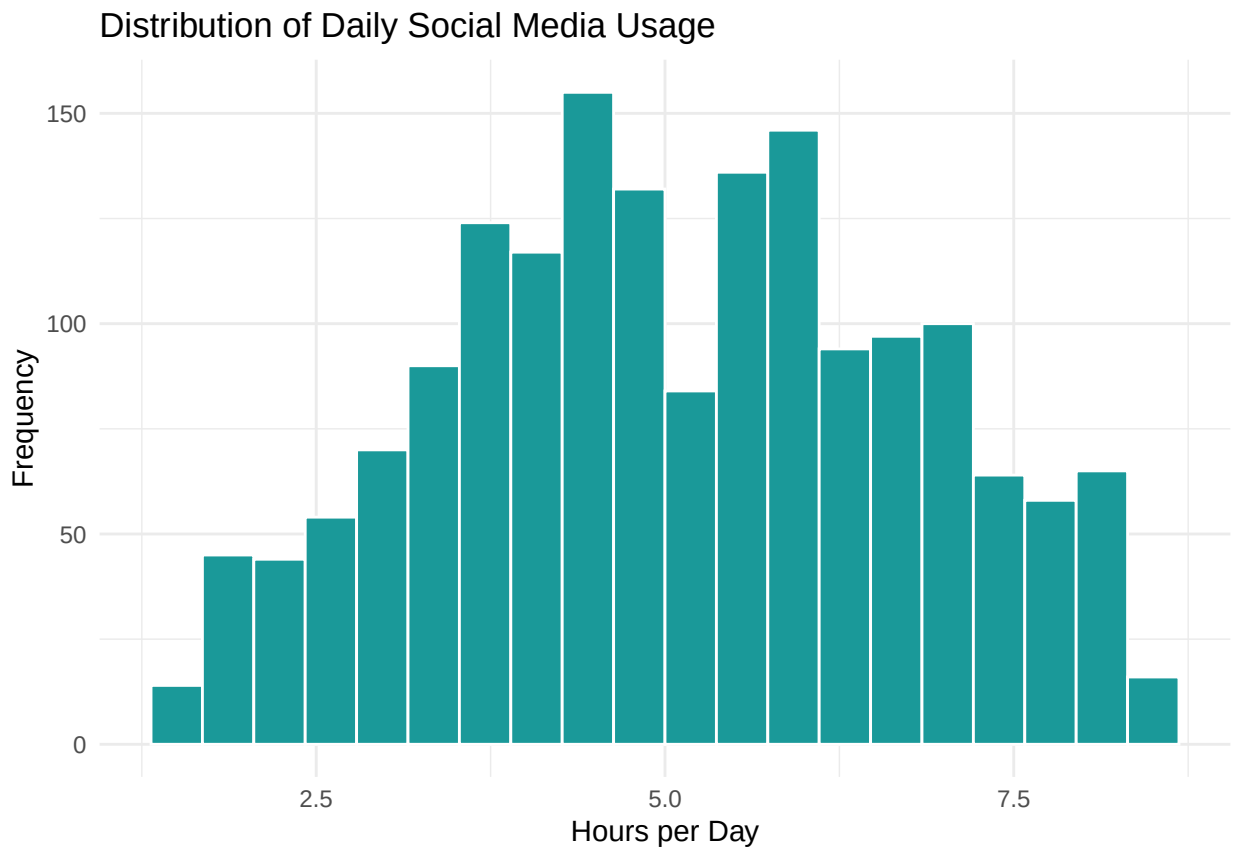

```
p2 <- ggplot(df, aes(x = Mental_Health_Score)) +
  geom_histogram(bins = 20, fill = "#0F6E6E", color = "white") +
  labs(title = "Distribution of Mental Health Score",
       x = "Score", y = "Frequency") +
  theme_minimal()
print(p2)
```




```
p3 <- ggplot(df, aes(x = Overall_Impact, y = Mental_Health_Score,
                     fill = Overall_Impact)) +
  geom_boxplot(alpha = 0.8) +
  scale_fill_manual(values = c("Positive" = "#0F6E6E",
                               "Neutral"  = "#E8A838",
                               "Negative" = "#8B0000")) +
  labs(title = "Mental Health Score by Impact Category",
       x = "Impact", y = "Score") +
  theme_minimal() + theme(legend.position = "none")
print(p3)
```




```
p4 <- ggplot(df, aes(x = Avg_Daily_Usage_Hours)) +
  geom_histogram(bins = 20, fill = "#1A9999", color = "white") +
  labs(title = "Distribution of Daily Social Media Usage",
       x = "Hours per Day", y = "Frequency") +
  theme_minimal()
print(p4)
```




```
df <- df[ , !(names(df) %in% "Student_ID")]
df$Gender_enc <- ifelse(df$Gender == "Male", 1, 0)

df$Academic_enc <- ifelse(df$Academic_Level == "High School", 1,
                   ifelse(df$Academic_Level == "Undergraduate", 2, 3))

df$Platform_enc <- as.integer(factor(df$Most_Used_Platform))

df$AcadPerf_enc <- ifelse(df$Affects_Academic_Performance == "Yes", 1, 0)

feature_cols <- c("Age", "Gender_enc", "Academic_enc", "Platform_enc",
                  "AcadPerf_enc", "Avg_Daily_Usage_Hours", "Sleep_Hours_Per_Night")

X <- df[ , feature_cols]
print(head(X))
```

```
##   Age Gender_enc Academic_enc Platform_enc AcadPerf_enc Avg_Daily_Usage_Hours
## 1  21          1            2           1            0                   4.0
## 2  23          0            2           5            0                   1.6
## 3  22          1            3           2            0                   4.6
## 4  18          1            2           6            1                   7.0
## 5  24          0            1           1            1                   7.5
## 6  19          0            1           8            1                   4.1
##   Sleep_Hours_Per_Night
## 1                   6.7
```

```
## 2                   8.6
## 3                   6.7
## 4                   5.4
## 5                   5.0
## 6                   6.8
```

```
y_class <- factor(df$Overall_Impact,
                  levels = c("Negative", "Neutral", "Positive"))
y_reg   <- df$Mental_Health_Score

train_idx     <- createDataPartition(y_class, p = 0.8, list = FALSE)
X_train       <- X[train_idx, ]
X_test        <- X[-train_idx, ]
y_class_train <- y_class[train_idx]
y_class_test  <- y_class[-train_idx]
y_reg_train   <- y_reg[train_idx]
y_reg_test    <- y_reg[-train_idx]

print(table(y_class_train))
```

```
## y_class_train
## Negative  Neutral Positive
##      752      214      400
```

```
ctrl <- trainControl(method = "cv", number = 5)

fit_lr <- train(x = X_train, y = y_class_train,
                method = "multinom",
                trControl = ctrl,
                trace = FALSE)
pred_lr   <- predict(fit_lr, X_test)
cm_lr     <- confusionMatrix(pred_lr, y_class_test)
acc_lr    <- round(cm_lr$overall["Accuracy"], 4)
f1_lr     <- round(mean(cm_lr$byClass[, "F1"], na.rm = TRUE), 4)

fit_rf_clf <- train(x = X_train, y = y_class_train,
                    method = "rf",
                    trControl = ctrl,
                    tuneGrid = expand.grid(mtry = c(2, 3, 4)),
                    ntree = 300)
pred_rf_clf   <- predict(fit_rf_clf, X_test)
cm_rf_clf     <- confusionMatrix(pred_rf_clf, y_class_test)
acc_rf_clf    <- round(cm_rf_clf$overall["Accuracy"], 4)
f1_rf_clf     <- round(mean(cm_rf_clf$byClass[, "F1"], na.rm = TRUE), 4)
cat("Accuracy:", acc_rf_clf, "\n")
```

```
## Accuracy: 0.9587
```

```
cat("Macro F1:", f1_rf_clf, "\n")
```

```
## Macro F1: 0.9442
```

```r
xgb_grid <- expand.grid(
  nrounds          = c(100, 200),
  max_depth        = c(3, 5),
  eta              = c(0.05, 0.1),
  gamma            = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  subsample        = 0.8
)
fit_xgb_clf <- train(x = X_train, y = y_class_train,
                     method = "xgbTree",
                     trControl = ctrl,
                     tuneGrid = xgb_grid)
```

```
## [19:02:51] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:52] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:52] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:52] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:52] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:53] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:53] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:53] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:53] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:53] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:54] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:54] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:54] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:54] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:55] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:55] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:55] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:55] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:55] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:02:56] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
```

```r
pred_xgb_clf  <- predict(fit_xgb_clf, X_test)
cm_xgb_clf    <- confusionMatrix(pred_xgb_clf, y_class_test)
acc_xgb_clf   <- round(cm_xgb_clf$overall["Accuracy"], 4)
f1_xgb_clf    <- round(mean(cm_xgb_clf$byClass[, "F1"], na.rm = TRUE), 4)

clf_table <- data.frame(
  Model    = c("Logistic Regression", "Random Forest", "XGBoost"),
  Accuracy = c(acc_lr, acc_rf_clf, acc_xgb_clf),
  Macro_F1 = c(f1_lr,  f1_rf_clf,  f1_xgb_clf)
)
print(clf_table)
```

```
##                 Model Accuracy Macro_F1
## 1 Logistic Regression   0.8348   0.7512
## 2       Random Forest   0.9587   0.9442
## 3             XGBoost   0.9587   0.9440
```

```r
best_clf <- clf_table$Model[which.max(clf_table$Macro_F1)]

best_pred_clf <- switch(best_clf,
  "Logistic Regression" = pred_lr,
  "Random Forest"       = pred_rf_clf,
  "XGBoost"             = pred_xgb_clf
)
print(confusionMatrix(best_pred_clf, y_class_test)$table)
```

```
##             Reference
## Prediction Negative Neutral Positive
##   Negative      181       5        1
##   Neutral         5      46        0
##   Positive        1       2       98
```

```r
ctrl_reg <- trainControl(method = "cv", number = 5)

baseline_rmse <- round(sqrt(mean((mean(y_reg_train) - y_reg_test)^2)), 4)

fit_lm <- train(x = X_train, y = y_reg_train,
                method = "lm",
                trControl = ctrl_reg)
pred_lm   <- predict(fit_lm, X_test)
rmse_lm   <- round(sqrt(mean((pred_lm - y_reg_test)^2)), 4)
r2_lm     <- round(cor(pred_lm, y_reg_test)^2, 4)

fit_rf_reg <- train(x = X_train, y = y_reg_train,
                    method = "rf",
                    trControl = ctrl_reg,
                    tuneGrid = expand.grid(mtry = c(2, 3, 4)),
                    ntree = 300)
pred_rf_reg <- predict(fit_rf_reg, X_test)
rmse_rf_reg <- round(sqrt(mean((pred_rf_reg - y_reg_test)^2)), 4)
r2_rf_reg   <- round(cor(pred_rf_reg, y_reg_test)^2, 4)

fit_xgb_reg <- train(x = X_train, y = y_reg_train,
                     method = "xgbTree",
                     trControl = ctrl_reg,
                     tuneGrid = xgb_grid)
```

```
## [19:03:06] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:06] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:06] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
```

```
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:07] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:08] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:08] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
## [19:03:08] WARNING: src/c_api/c_api.cc:935: `ntree_limit` is deprecated, use `iteration_range` instea
```

```
pred_xgb_reg <- predict(fit_xgb_reg, X_test)
rmse_xgb_reg <- round(sqrt(mean((pred_xgb_reg - y_reg_test)^2)), 4)
r2_xgb_reg   <- round(cor(pred_xgb_reg, y_reg_test)^2, 4)


reg_table <- data.frame(
  Model = c("Mean Baseline", "Linear Regression", "Random Forest", "XGBoost"),
  RMSE  = c(baseline_rmse, rmse_lm, rmse_rf_reg, rmse_xgb_reg),
  R2    = c(NA, r2_lm, r2_rf_reg, r2_xgb_reg)
)

print(reg_table)
```

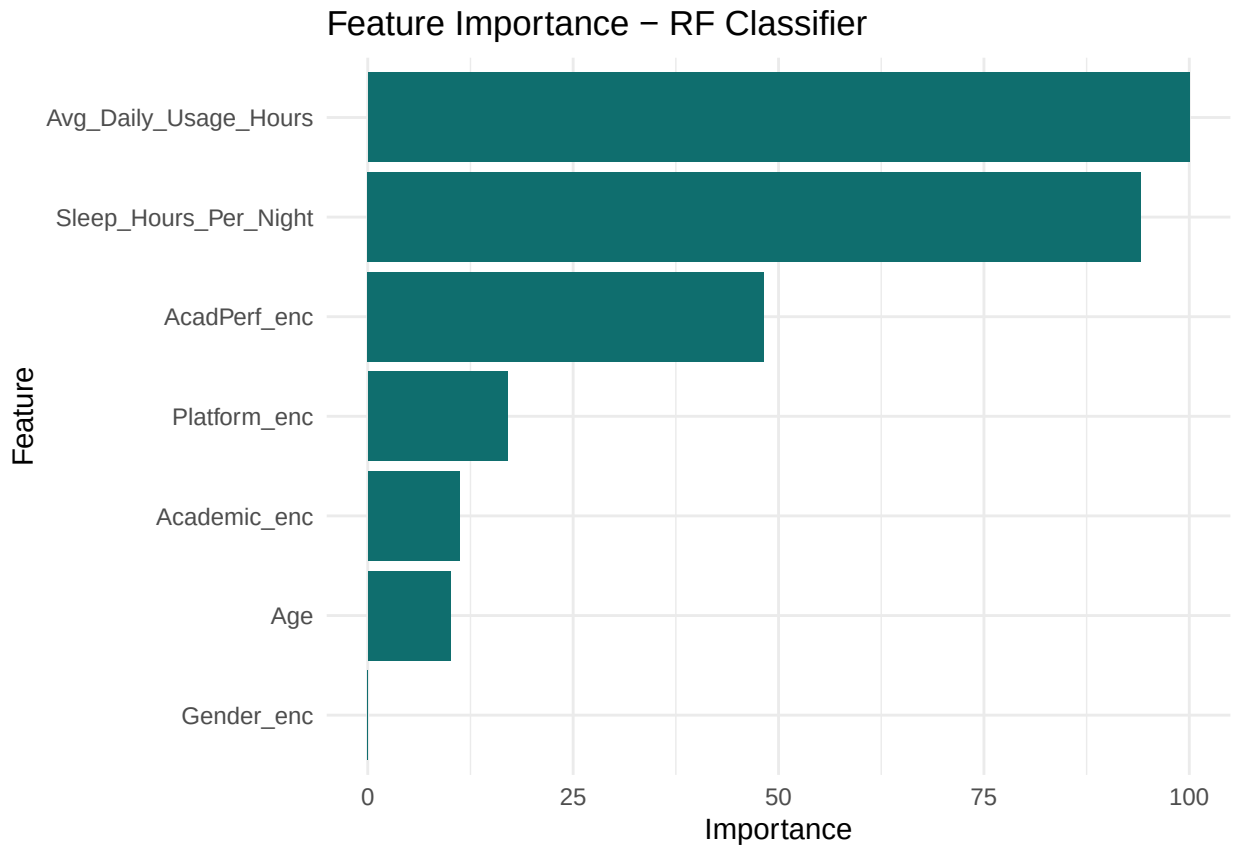
```
##               Model   RMSE     R2
## 1     Mean Baseline 1.2740     NA
## 2 Linear Regression 0.6436 0.7458
## 3     Random Forest 0.5279 0.8296
## 4           XGBoost 0.5446 0.8177
```

```
best_reg <- reg_table$Model[which.min(reg_table$RMSE)]

imp_clf <- varImp(fit_rf_clf)$importance
imp_clf$Feature <- rownames(imp_clf)
imp_clf <- imp_clf[order(-imp_clf$Overall), ]
print(imp_clf)
```

```
##                         Overall               Feature
## Avg_Daily_Usage_Hours 100.00000 Avg_Daily_Usage_Hours
## Sleep_Hours_Per_Night  94.09631 Sleep_Hours_Per_Night
## AcadPerf_enc           48.25459          AcadPerf_enc
## Platform_enc           17.01702          Platform_enc
## Academic_enc           11.15381          Academic_enc
## Age                    10.08907                   Age
## Gender_enc              0.00000            Gender_enc
```

```
p5 <- ggplot(imp_clf, aes(x = reorder(Feature, Overall), y = Overall)) +
  geom_col(fill = "#0F6E6E") +
  coord_flip() +
  labs(title = "Feature Importance - RF Classifier",
       x = "Feature", y = "Importance") +
  theme_minimal()
print(p5)
```




```
imp_reg <- varImp(fit_xgb_reg)$importance
imp_reg$Feature <- rownames(imp_reg)
imp_reg <- imp_reg[order(-imp_reg$Overall), ]
cat("\nTop features (XGB Regressor):\n")
```

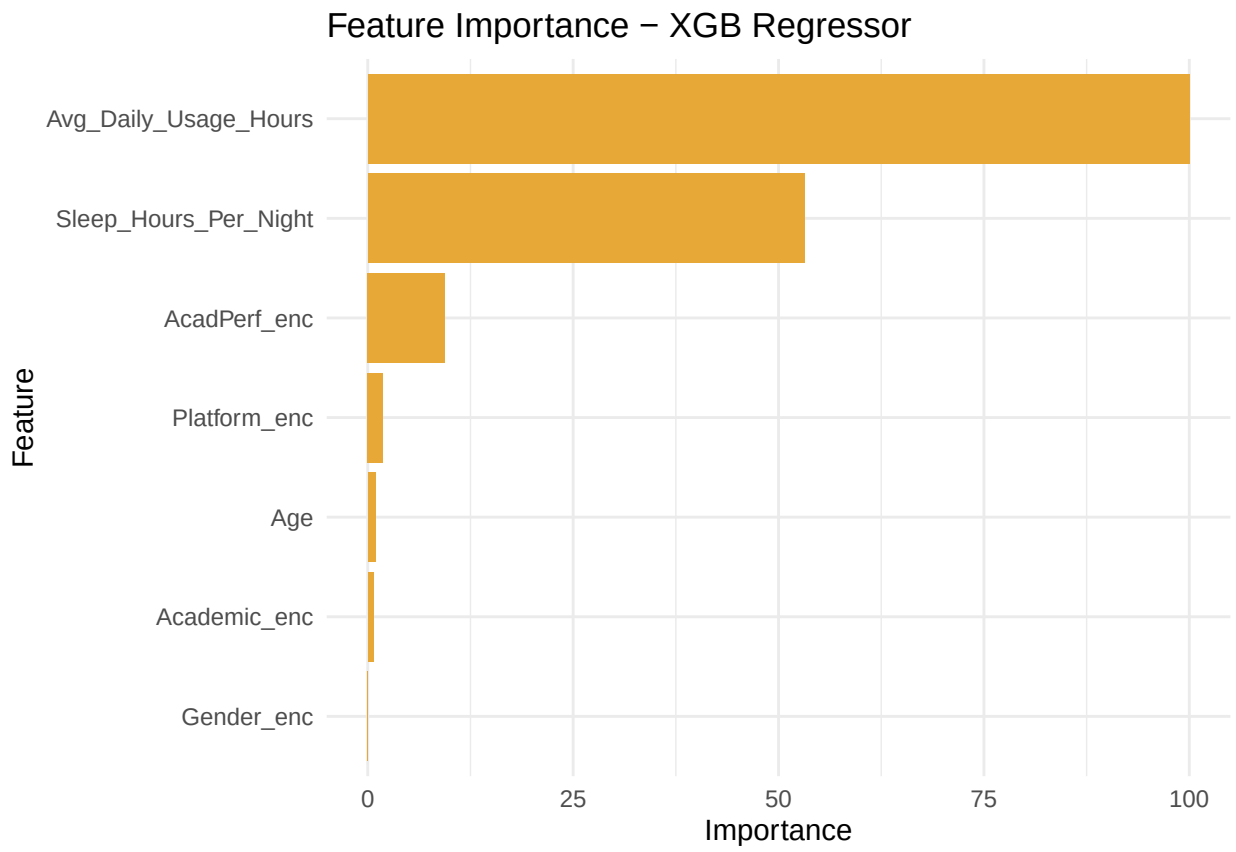
```
##
## Top features (XGB Regressor):
```

```
print(imp_reg)
```

```
##                           Overall               Feature
## Avg_Daily_Usage_Hours 100.0000000 Avg_Daily_Usage_Hours
## Sleep_Hours_Per_Night  53.1713917 Sleep_Hours_Per_Night
## AcadPerf_enc            9.3990004          AcadPerf_enc
## Platform_enc            1.8290144          Platform_enc
## Age                     0.9421507                   Age
## Academic_enc            0.6755554          Academic_enc
## Gender_enc              0.0000000            Gender_enc
```

```
p6 <- ggplot(imp_reg, aes(x = reorder(Feature, Overall), y = Overall)) +
  geom_col(fill = "#E8A838") +
  coord_flip() +
  labs(title = "Feature Importance - XGB Regressor",
       x = "Feature", y = "Importance") +
```

```
  theme_minimal()
print(p6)
```




```
for (i in 1:nrow(clf_table)) {
  cat(sprintf("  %-22s  Accuracy: %s%%   Macro F1: %s\n",
              clf_table$Model[i],
              round(clf_table$Accuracy[i] * 100, 1),
              clf_table$Macro_F1[i]))
}
```

```
##   Logistic Regression     Accuracy: 83.5%   Macro F1: 0.7512
##   Random Forest           Accuracy: 95.9%   Macro F1: 0.9442
##   XGBoost                 Accuracy: 95.9%   Macro F1: 0.944
```

```
for (i in 1:nrow(reg_table)) {
  cat(sprintf("  %-22s  RMSE: %s   R2: %s\n",
              reg_table$Model[i],
              reg_table$RMSE[i],
              ifelse(is.na(reg_table$R2[i]), "-", reg_table$R2[i])))
}
```

```
##   Mean Baseline           RMSE: 1.274   R2: -
##   Linear Regression       RMSE: 0.6436   R2: 0.7458
##   Random Forest           RMSE: 0.5279   R2: 0.8296
##   XGBoost                 RMSE: 0.5446   R2: 0.8177
```

```r
cat("\n[Best Classification model] →", best_clf, "\n")
```

```
##
## [Best Classification model] → Random Forest
```

```r
cat("[Best Regression model]     →", best_reg, "\n")
```

```
## [Best Regression model]     → Random Forest
```

```r
cat("\n[Top 3 features for slide bar chart]\n")
```

```
##
## [Top 3 features for slide bar chart]
```

```r
cat("  Classification:", paste(head(imp_clf$Feature, 3), collapse = ", "), "\n")
```

```
##   Classification: Avg_Daily_Usage_Hours, Sleep_Hours_Per_Night, AcadPerf_enc
```

```r
cat("  Regression:    ", paste(head(imp_reg$Feature, 3), collapse = ", "), "\n")
```

```
##   Regression:     Avg_Daily_Usage_Hours, Sleep_Hours_Per_Night, AcadPerf_enc
```